AI Enhancement Documentation

Overview

The GitHub Stars Organizer has been enhanced with AI-powered repository categorization using Gemma 3-270m via Ollama. This enhancement provides intelligent categorization suggestions while maintaining user control as the primary interaction method.

✨ Key Features

🤖 AI-Powered Categorization

- **Local AI Processing**: Uses Ollama with Gemma 3-270m running locally
- **Zero-Shot Prompting**: Analyzes repositories without training examples
- **Intelligent Suggestions**: Considers repository metadata, language, topics, and description
- **Confidence Scoring**: Provides confidence levels for each suggestion

👤 User Control First

- **Non-Destructive**: AI never automatically applies changes
- **Manual Override**: Drag-and-drop remains the primary interaction method
- **Review Process**: Users review and approve/reject each suggestion
- **One-Click Undo**: Complete undo system for all AI actions

⚡ Performance Optimized

- **Batch Processing**: Handles multiple repositories efficiently

- **Progress Tracking**: Visual indicators for long-running operations
- **Configurable Timeouts**: Prevents hanging on slow responses
- **Error Recovery**: Graceful fallback when AI is unavailable

🏗️ Architecture

Backend (Convex)

Database Schema Extensions

```
aiCategorizationSuggestions: {
  userId: Id<"users">
  repositoryId: Id<"repositories">
  suggestedCategoryName: string
  confidence: number (0-1)
  status: "pending" | "accepted" | "rejected" | "applied"
  reasoning: string
  // ... metadata
}

categorizationHistory: {
  userId: Id<"users">
  action: "ai_batch_categorize" | "undo" | "redo"
  repositoryIds: Id<"repositories">[]
  beforeState: object // For undo functionality
  afterState: object
  aiGenerated: boolean
  // ... metadata
}

aiSettings: {
  userId: Id<"users">
  ollamaEndpoint: string
  enableAI: boolean
  aiModel: string
  categoryPrimer: object // Few-shot examples
  advancedSettings: object // Timeouts, batch sizes, etc.
}
```

Core Functions

- `generateSingleRepositorySuggestion`: Analyze one repository
- `generateBatchRepositorySuggestions`: Process multiple repositories
- `testOllamaConnection`: Verify AI service availability

- `applyAcceptedSuggestions`: Apply user-approved suggestions
- `undoLastCategorization`: Revert AI-driven changes

Frontend (React Native)

Components

- `AIOrganizeButton`: Main trigger for AI analysis
- `AISuggestionsReview`: Modal for reviewing suggestions
- `AIUndoButton`: Quick undo for recent AI actions
- `AISettingsScreen`: Configuration interface

User Flow

1. User clicks "Have Gemma Organize"
2. System tests Ollama connection
3. Repositories analyzed in batches
4. Suggestions presented for review
5. User accepts/rejects individual suggestions
6. Accepted suggestions applied with undo capability

🛠️ Setup Instructions

Prerequisites

1. **Install Ollama**
   ```bash
   # macOS
   brew install ollama

# Linux
curl -fsSL https://ollama.ai/install.sh | sh
   ```

Windows - Download from ollama.ai
```

1. **Download Gemma Model**
   `bash ollama pull gemma:2b # or for better quality (requires more RAM): ollama pull gemma:7b`
2. **Start Ollama Server**
   `bash ollama serve # Default endpoint: http://localhost:11434`

## Configuration

1. **Enable AI Features**
   - Open the app
   - Navigate to "AI Settings" from the main screen
   - Toggle "Enable AI Categorization"
2. **Configure Ollama Connection**
   - Set Ollama Endpoint (default: `http://localhost:11434`)
   - Set AI Model (default: `gemma:2b`)
   - Test connection to verify setup
3. **Advanced Settings** (Optional)
   - Request Timeout: 30000ms (30 seconds)
   - Batch Size: 5 repositories
   - Confidence Threshold: 0.6 (60%)
   - Max Retries: 3 attempts

# 📱 Usage Guide

## Basic AI Categorization

1. **Sync Your Repositories**
   - Ensure your GitHub repositories are synced
   - Go to "Sync" tab if needed
```

2. **Start AI Analysis**
   - On the main screen, tap "Have Gemma Organize"
   - Confirm the analysis dialog
   - Wait for processing (may take 2-5 minutes for many repositories)
3. **Review Suggestions**
   - Review modal opens automatically when complete
   - Each suggestion shows:
     - Repository name and language
     - Suggested category with color/icon
     - Confidence level (High/Medium/Low)
     - AI reasoning for the suggestion
4. **Accept/Reject Suggestions**
   - Tap "Accept" for good suggestions
   - Tap "Reject" for poor suggestions
   - Use "Accept All" or "Reject All" for bulk actions
5. **Apply Changes**
   - Tap "Apply X Suggestions" to create categories
   - New categories are created automatically
   - Repositories are organized into suggested categories

Undo System

1. **Immediate Undo**
   - "Undo AI Organization" button appears after AI actions
   - Shows what was changed and when
   - One-click restore to previous state
2. **Complete Restoration**
   - Removes all AI-created category assignments
   - Restores previous manual organization
   - Cannot be reversed (but you can re-run AI analysis)

Manual Override

- **Drag & Drop**: Always available and preferred method
- **Category Creation**: Manual categories work alongside AI suggestions
- **Mixed Usage**: Combine AI suggestions with manual organization

🔧 Advanced Configuration

Prompt Engineering

The AI uses zero-shot prompting by default:

```
Act as an experienced developer with advanced repository
categorization skills.
Analyze this GitHub repository and suggest how to organize it.

Repository Details:
- Name: [repo name]
- Description: [description]
- Language: [language]
- Topics: [topics]
- Stars: [count]

Instructions:
1. Analyze the repository's purpose, technology stack, and domain
2. Suggest either an existing category or propose a new one
3. Provide a confidence score (0.0 to 1.0)
4. Give a brief reasoning

Response Format (JSON only):
{
  "categoryName": "Category Name",
  "confidence": 0.85,
  "reasoning": "Brief explanation",
  "isNewCategory": true,
  "suggestedColor": "#3B82F6",
  "suggestedIcon": "code"
}
```

Performance Tuning

Batch Size

- **Small (1-3)**: Slower but more reliable
- **Medium (5-7)**: Balanced performance

- **Large (10+)**: Faster but may timeout

Timeout Settings

- **Conservative**: 45000ms (45 seconds)
- **Standard**: 30000ms (30 seconds)
- **Aggressive**: 15000ms (15 seconds)

Model Selection

- **gemma:2b**: Fast, lightweight, good accuracy
- **gemma:7b**: Better accuracy, requires 8GB+ RAM
- **llama2:7b**: Alternative model option

Troubleshooting

Connection Issues

```
# Check if Ollama is running
curl http://localhost:11434/api/tags

# Restart Ollama service
killall ollama
ollama serve
```

Model Issues

```
# Re-download model
ollama rm gemma:2b
ollama pull gemma:2b

# List available models
ollama list
```

Performance Issues

- Reduce batch size in settings
- Increase timeout duration
- Use smaller model (gemma:2b vs gemma:7b)
- Close other applications to free RAM

🔒 Privacy & Security

Data Privacy

- **Local Processing**: All AI analysis happens on your device
- **No External Calls**: Repository data never sent to external services
- **Ollama Only**: Only communicates with your local Ollama instance

Security Considerations

- **Network Access**: App only accesses local Ollama endpoint
- **Token Security**: GitHub tokens remain encrypted in Convex
- **Data Isolation**: Each user's AI settings and suggestions are isolated

📊 Monitoring & Analytics

Built-in Metrics

- Processing time per repository
- Average confidence scores
- Success/failure rates
- User acceptance rates for suggestions

Logging

- Connection attempts and results
- Processing job status and errors
- User actions (accept/reject/undo)

🚀 Future Enhancements

Planned Features

- **Few-Shot Learning**: Learn from user corrections
- **Category Merger**: Suggest consolidating similar categories
- **Smart Filters**: AI-powered search and filtering
- **Batch Operations**: Multi-select repositories for AI analysis

Potential Improvements

- **Custom Prompts**: User-defined categorization rules
- **Multiple Models**: Support for different AI models
- **Offline Mode**: Cached suggestions for offline use
- **Export/Import**: Share categorization patterns

🤝 Contributing

Adding New Features

1. Backend changes go in `convex/ai.ts`
2. Frontend components in `src/components/AI*.tsx`
3. Update schema in `convex/schema.ts` if needed
4. Add settings to `app/ai-settings.tsx`

Testing

1. Test with various repository types
2. Verify undo functionality works correctly
3. Check error handling for network issues
4. Validate settings persistence

📞 Support

For issues or questions about the AI enhancement:
1. Check the troubleshooting section
2. Verify Ollama setup and model availability
3. Test connection in AI Settings
4. Review app logs for error details

Remember: The AI is designed to assist, not replace your judgment. Manual categorization via drag-and-drop remains the primary and most flexible method for organizing your repositories.